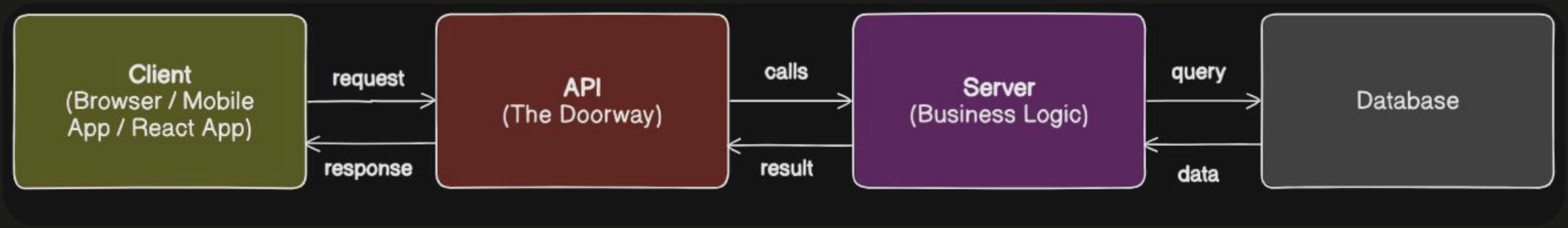
Fetch API in JavaScript

Lec-15

Made with GAMMA

What is an API

- **API** Stands for- **Application Programming Interface**.
- It is a **defined way for one piece of software to ask another piece of software to do something** (or to give it some data) and to get an answer back, **without needing to know how that other software works inside**.
- **Application**: Any software that does a job: a weather service, a payment system, your own backend.
- **Programming**: The API is used by programs (code written by developers), not by end users clicking buttons.
- **Interface**: The meeting point between two systems. Like a contract: "if you send me this, I promise to reply with that."
- **Client only talks to the API**, it never touches the server internals or database directly.



Made with GAMMA

Types of API

- API is categorised into **two** types, **based on who can use them** and **based on style/protocol**.

Category-1: By who can use them

Type	Who can access	Example
Public / Open	Any Developer (often needs a free API key)	OpenWeather, PokeAPI, GitHub public API
Partner	Selected business partners under agreement	A bank exposing an API to approved fintech apps.
Private / Internal	Only inside one company	Your comapny's backend used by it's own web and mobile app
Composite	Combines severak API calls into one.	One endpoint which fetches user + order + payment together.

Category-2: Based on Style/ Protocol

Style	What it is	Data Format	Typical use
REST	Resources identified by URL's, manipulated with HTTP methods. The most common style.	JSON (mostly)	Almost every web/mobile backend.
GraphQL	One endpoint; the client writes a query describing exactly which field it wants.	JSON	Complex UI's, mobile apps needing fewer round trips.
SOAP	Older, strict, XML-based protocol with formal contracts (WSDL).	XML	Legacy enterprise, banking, telecom systems
gRPC	High-performance remote procedure calls over HTTP/2 using Protocol Buffers.	Binary (protobuf)	Service-to-service calls inside microservices.
WebSocket	A persistent two-way connection; both sides can send anytime.	Any (often JSON)	Chat, live dashboards, multiplayer games.
Webhook	Server calls your URL when an event happens ("reverse API").	JSON	Payment confirmations, GitHub push events.

Made with GAMMA

HTTP Methods

- The **method tells the server what action you want to perform on the resource in the URL**.

Method	Purpose
GET	Read/Fetch data
POST	Create a new resource, or trigger an action
PUT	Replace a resource completely
PATCH	Update only some fields
DELETE	Remove a resource
HEAD	Same as GET, but only headers, no body.
OPTIONS	Ask which methods are allowed (CORS preflight)

Made with GAMMA

HTTP Status Codes

- Every response starts with a three-digit status code so the client knows immediately what happened, before reading the body.
- **The first digit tells the category**



Code	Name	When to use it
200	OK	Request succeeded (GET, PUT, PATCH). Body contains the result.
201	Created	A new resource was created (after POST), Add a ***Location*** header.
204	No Content	Success, nothing to return (commonly after DELETE).
301 / 302	Moved Permanently / Found	Redirect to another URL.
304	Not Modified	Cached copy is still valid; no body sent.
400	Bad Request	Malformed request or invalid input (bad JSON, missing field).
401	Unauthorized	**Not authenticated**: no token, bad token or expired token. "Who are you?"
403	Forbidden	**Authenticated but not allowed.** "I know you, but you cannot do this."
404	Not Found	Resource does not exist.
405	Method Not Allowed	URL exists but not for this method (eg. DELETE on a read-only-route)
409	Conflict	Conflicts with current state, eg email already registered.
422	Unprocessable Entity	JSON is well-formed but fails validation rules.
429	Too Many Requests	Rate limit exceeded. Check the ***Retry-After*** header.
500	Internal Server Error	Unexpected bug on the server.
502 / 503 / 504	Bad Gateway/ Unavailable / Gateway Timeout	Upstream service failed, server overloaded or down, or upstream too slow.

Made with GAMMA

Fetch API

- The **Fetch API**, lets JavaScript **download resources** from a web server.
- It **provides an interface for fetching (sending/receiving) resources**.
- It can be **used to load text files, JSON data, images** and many other type of resources.
- The **fetch() method is used to fetch a resource (data)**.
- The **fetch() method returns a promise and that resolves to a response object**.
- **Syntax**:

```
let promise = fetch(url,[options])
```

- **Eg: Suppose we're fetching a response from the following api url**

```
let apiLink = "https://randomuser.me/api/"
```

As we know **fetch()** method will return a promise, so in order to handle it we will use async await inside a try and catch block.

```
const sportsApiReq = async ()=>
{
 try {
     console.log('Getting Data...');
     let response = await fetch(apiLink)
     console.log(response);
    }
 catch (error)
 {
    console.log(error);
 }
}
```

```
Getting Data...                                    fetch-demo.html:22
                                                   fetch-demo.html:24
▸ Response {type: 'cors', url:
  'https://randomuser.me/api/', redirected: false, statu
  s: 200, ok: true, …}
>
```

The response object can have following properties/ methods:

Property/ Method	Description
ok	True if request succeded
status	The HTTP status code
statustext	The HTTP status message
text()	Reads the response as text
json()	Reads the response as JSON
blob()	Reads the response as binary
bytes()	Reads the response as bytes
arrayBuffer()	Reads the response as an Array buffer

Promise.all()

- Used to fetch data from multiple sources at once.

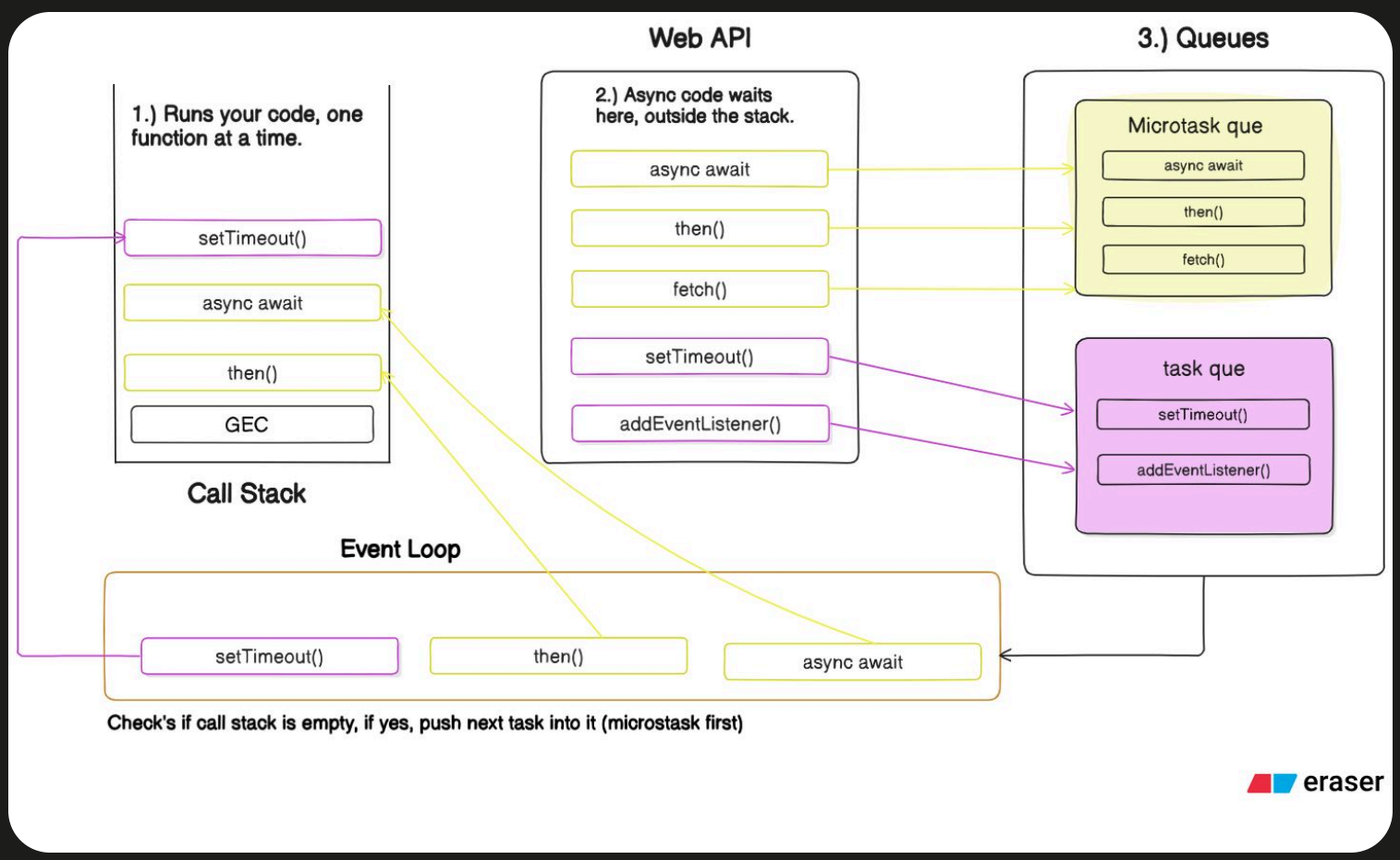
```
async function userAndCountry(){
  try{
     const [userResponse, countryResponse] = await Promise.all([fetch(apiLink1), fetch(apiLink2)])
     const JsonUserResponse = await userResponse.json()
     const JsonCountryResponse = await countryResponse.json()

     console.log(JsonUserResponse);
     console.log(JsonCountryResponse);
    }catch(error){
          console.log(error);
       }
 }
userAndCountry()
```

Made with GAMMA

Event Loop

- The event loop **keeps checking whether the call stack is empty or not**.
- If it is empty, **it takes the next waiting callback from a queue and pushes it onto the stack**.
- The working of event loop are as follows:



Let's understand the whole diagram in 4 simple steps:

Step-1 - Run on the call stack : Your code executes one function at a time. Synchronous code, like console.log(1), runs straight away.

Step-2 - Run on the call stack : Calls like **setTimeout, fetch and click handlers are given to the browser**, which **waits on them outside the stack**, so nothing blocks.

Step-3 - Queue the callback when it's ready : Promise **callbacks (.then) go into the microtask queue**. **Timer and event callbacks go into the task queue**.

Step-4 - The event loop moves callbacks to the stack : When the **call stack is empty**, the **loop pushes the next waiting callback onto it**, always emptying the microtask queue before taking the next task.

Made with GAMMA